CHAPTER 5
Understanding the Shell

IN THIS CHAPTER

Investigating shell types

Understanding the parent/child shell relationship

Using subshells creatively

Investigating built-in shell commands

Now that you know a few shell basics, such as reaching the shell and rudimentary commands, it's time to explore the actual shell process. To understand it, you need to know how it operates in different circumstances.

A shell is not just a CLI. It is a complicated interactive running program. Entering commands and using the shell to run scripts can raise some interesting and confusing issues. Understanding the shell process and its relationships helps you resolve these issues or avoid them altogether.

This chapter takes you through learning about the shell process and how it operates in various situations. We'll explore how subshells are created as well as the relationship to their parent shell. The different commands that create child processes are examined along with those that don't (built-in commands). We also cover some shell tips and tricks you can try to make your CLI experience more productive.

Investigating Shell Types

The shell program that the system starts, when you log into the system, depends on your user ID configuration. In the `/etc/passwd` file, the user ID has its default shell program listed in field #7 of its entry. This default shell program is started when the user either logs into a virtual console terminal or starts a terminal emulator in the GUI.

In the following example, the user `christine` has the GNU Bash shell as their default shell program:

```
$ cat /etc/passwd
[...]
christine:x:1001:1001::/home/christine:/bin/bash
$
```

The Bash shell program (`bash`) typically resides in the `/usr/bin` directory on modern Linux systems. However, on your Linux system you may find it in the `/bin` directory. The `which bash` command can help here by providing the directory and filename to use for the Bash shell:

```
$ which bash
/usr/bin/bash
$
```

A long listing reveals that the `bash` file (the Bash shell) is an executable program via the trailing asterisk (`*`) on the file's name:

```
$ ls -lF /usr/bin/bash
-rwxr-xr-x. 1 root root 1219248 Nov  8 11:30 /usr/bin/bash*
$
```

> **NOTE**
>
> **Typically on modern Linux systems, the `/bin` directory is symbolically linked to the `/usr/bin/` directory, which is why the user `christine` has `/bin/bash` listed as their default shell program, but the Bash shell program actually resides in the `/usr/bin/` directory. Symbolic (soft) links were covered in Chapter 3, "Basic Bash Shell Commands."**

Several other shell programs are on this particular Linux system. They include tcsh, which is based on the original C shell:

```
$ which tcsh
/usr/bin/tcsh
$ ls -lF /usr/bin/tcsh
-rwxr-xr-x. 1 root root 465200 May 14  2019 /usr/bin/tcsh*
$
```

Another shell on this system is zsh, which is a more elaborate version of the Bash shell. It also has a few tcsh features as well as other elements:

```
$ which zsh
/usr/bin/zsh
$ ls -lF /usr/bin/zsh
-rwxr-xr-x. 1 root root 879872 May 11  2019 /usr/bin/zsh*
$
```

> **TIP**
>
> **If you don't find some of these shells on your Linux system, you may be able to install them. Chapter 9, "Installing Software," can help you accomplish this task.**

A soft link (see Chapter 3) of the C shell points to the tcsh shell:

```
$ which csh
/usr/bin/csh
$ ls -lF /usr/bin/csh
lrwxrwxrwx. 1 root root 4 May 14  2019 /usr/bin/csh -> tcsh*
$
```

On Debian-based Linux systems, such as Ubuntu, you often find `dash` , which is a version of the Ash shell:

```
$ which dash
/usr/bin/dash
$ ls -lF /usr/bin/dash
-rwxr-xr-x 1 root root 129816 Jul 18  2019 /usr/bin/dash*
$
```

> **NOTE**
>
> **A brief description of various shells was included in Chapter 1, "Starting with Linux Shells." You may be interested in learning even more about shells other than the GNU Bash shell. Additional alternative shell information is in Chapter 23, "Working with Alternative Shells."**

On most Linux systems, you'll find the various installed shells that can be used as the user's default shell within the `/etc/shells` file, as shown here:

```
$ cat /etc/shells
/bin/sh
/bin/bash
/usr/bin/sh
/usr/bin/bash
/bin/csh
/bin/tcsh
/usr/bin/csh
/usr/bin/tcsh
/usr/bin/zsh
/bin/zsh
$
```

> **NOTE**
>
> **On many Linux distributions, you'll find that it appears as if a shell file exists in two places — `/bin` and `/usr/bin` . This is because on modern Linux systems, these two directories are often symbolically linked together, with `/bin` pointing to `/usr/bin` . And you can use either directory to access the shell. Symbolic links, also called soft links, were covered in Chapter 3.**

A user can set any of these different shell programs on this system as their shell. However, due to its popularity, it is rare to use any other shell instead of Bash as an account's default interactive shell. The *default interactive shell*, also called a *login shell*, starts whenever a user logs into a virtual console terminal or starts a terminal emulator in the GUI.

Another shell, `sh` , is the *default system shell*. The default system shell is used for system shell scripts, such as those needed at startup.

Often, you see a distribution with its default system shell (`sh`) pointing to the Bash shell using a soft link. An example of this is shown here on a CentOS distribution:

```
$ which sh
/usr/bin/sh
$ ls -l /usr/bin/sh
lrwxrwxrwx. 1 root root 4 Nov  8 11:30 /usr/bin/sh -> bash
$
```

However, be aware that on some distributions, the default system shell is linked differently, such as on this Ubuntu distribution:

```
$ which sh
/usr/bin/sh
$ ls -l /usr/bin/sh
lrwxrwxrwx 1 root root 4 Mar 10 18:43 /usr/bin/sh -> dash
$
```

In this case, the default system shell, `/usr/bin/sh` , points to the `Dash` shell, instead of the Bash shell.

> **TIP**
>
> **For Bash shell scripts, these two different shells, default interactive shell and default system shell, can cause problems. Be sure to read about the important syntax needed for a Bash shell script's first line in Chapter 11, "Basic Script Building," to avoid these issues.**

You are not forced to stick with your default interactive shell. You can start any shell available on your distribution simply by typing its name. However, there's no fanfare or message displayed to indicate what shell you are currently using. But help is available from the `$0` variable. The command `echo $0` will display the name of your current shell, providing the needed reference.

> **NOTE**
>
> **The `echo $0` command shows the current shell in use only when issued at a shell prompt. If used in a shell script, it will display the script's name instead. This is covered in Chapter 14, "Handling User Input."**

With our handy `$0` variable, we'll display the shell we are currently using, start the Dash shell by typing the command **`dash`**, and show the new shell's name via `echo $0` again:

```
$ echo $0
-bash
$
$ dash
$
$ echo $0
dash
$
```

> **NOTE**
>
> **In the previous example, notice the dash (-) in front of the `bash` response from the first `echo $0` command. This indicates that this shell is the user's login shell.**

The `$` prompt is a CLI prompt for the Dash shell. You can leave the Dash shell program (and the Bash shell for that matter) by typing the command **`exit`** :

```
$ echo $0
dash
$ exit
$ echo $0
-bash
$
```

Jumping back and forth through the various shells seems simple, but there is more to the action happening behind the scenes. To understand this process, the next section explores the relationship between a login shell program and a newly started shell program.

Exploring Parent and Child Shell Relationships

The default interactive shell (login shell) that starts when a user logs into a virtual console terminal or starts a terminal emulator in the GUI is a *parent shell*. As you have read so far in this book, a parent shell process provides a CLI prompt and waits for commands to be entered.

When the `bash` command (or other shell program name) is entered at the CLI prompt, a new shell program is created. This is a *child shell*. A child shell also has a CLI prompt and waits for commands to be entered.

Because you do not see any relevant messages when you type **`bash`** and spawn a child shell, another command can help bring clarity. The `ps` command was covered in Chapter 4, "More Bash Shell Commands." Using this with the `-f` option before and after entering a child shell is useful:

```
$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
christi+  6160  6156  0 11:01 pts/1    00:00:00 -bash
christi+  7141  6160  0 12:51 pts/1    00:00:00 ps -f
$
$ bash
$
$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
```

```
christi+  6160  6156  0 11:01 pts/1    00:00:00 -bash
christi+  7142  6160  0 12:52 pts/1    00:00:00 bash
christi+  7164  7142  0 12:52 pts/1    00:00:00 ps -f
$
```

The first use of `ps -f` shows two processes. One process has a process ID of 6160 (second column) and is running the Bash shell program (last column). The second process (process ID 7141) is the actual `ps -f` command running.

> **NOTE**
>
> **A process is a running program. The Bash shell is a program, and when it runs, it is a process. A running shell is simply one type of process. Therefore, when reading about running a Bash shell, you often see the word "shell" and the word "process" used interchangeably.**

In the previous example, after the command `bash` is entered, a child shell is created. The second `ps -f` is executed from within the child shell. From this display, you can see that *two* Bash shell programs are running. The first Bash shell program, the parent shell process, has the original process ID (PID) of 6160. The second Bash shell program, the child shell process, has a PID of 7142. Note that the child shell has a parent process ID (PPID) of 6160, denoting that the parent shell process is its parent. Figure 5-1 diagrams this relationship.

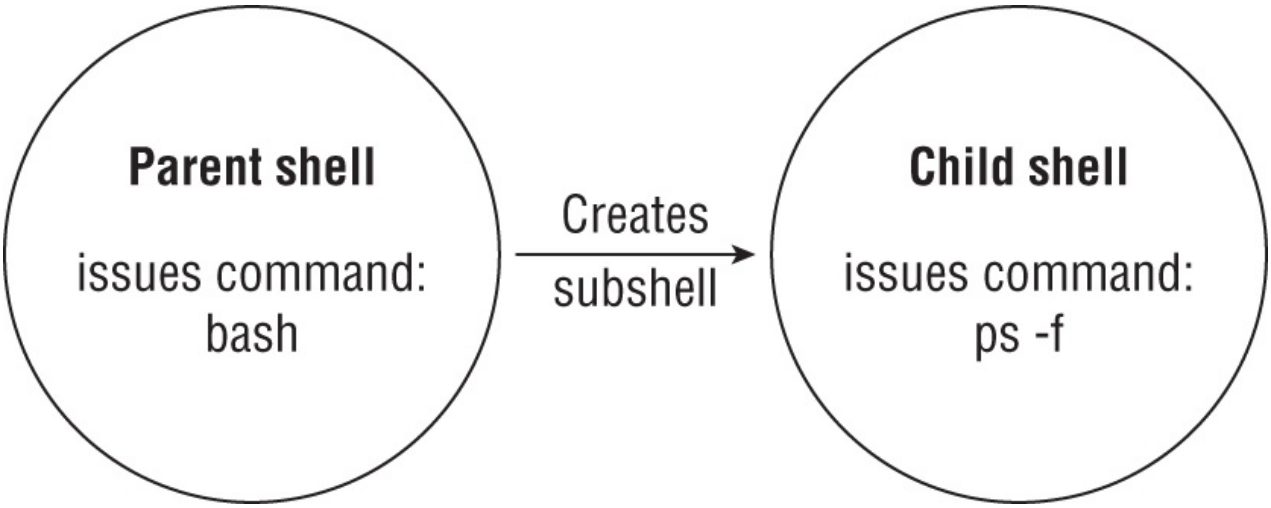

FIGURE 5-1 Parent and child Bash shell processes

When a child shell process is spawned, only some of the parent's environment is copied to the child's shell environment. This can cause problems with items such as variables. How to prevent such problems is covered in Chapter 6, "Using Linux Environment Variables."

A child shell is also called a *subshell*. A subshell can be created from a parent shell, and a subshell can be created from another subshell:

```
$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
christi+  7650  7649  0 16:01 pts/0    00:00:00 -bash
christi+  7686  7650  0 16:02 pts/0    00:00:00 ps -f
$
$ bash
$ bash
$ bash
$
$ ps --forest
  PID TTY          TIME CMD
 7650 pts/0    00:00:00 bash
 7687 pts/0    00:00:00  \_ bash
 7709 pts/0    00:00:00      \_ bash
 7731 pts/0    00:00:00          \_ bash
 7753 pts/0    00:00:00              \_ ps
$
```

In the preceding example, the `bash` command was entered three times. Effectively, this created three subshells. The `ps --forest` command shows the nesting of these subshells. Figure 5-2 also shows this subshell nesting.

The `ps -f` command can be useful in subshell nesting, because it displays who is whose parent via the PPID column:

```
$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
christi+  7650  7649  0 16:01 pts/0    00:00:00 -bash
christi+  7687  7650  0 16:02 pts/0    00:00:00 bash
christi+  7709  7687  0 16:02 pts/0    00:00:00 bash
christi+  7731  7709  0 16:02 pts/0    00:00:00 bash
christi+  7781  7731  0 16:04 pts/0    00:00:00 ps -f
```

```
$
```

The Bash shell program can use command-line options to modify the shell's start. Table 5-1 lists a few of these available switches to use with the `bash` command.

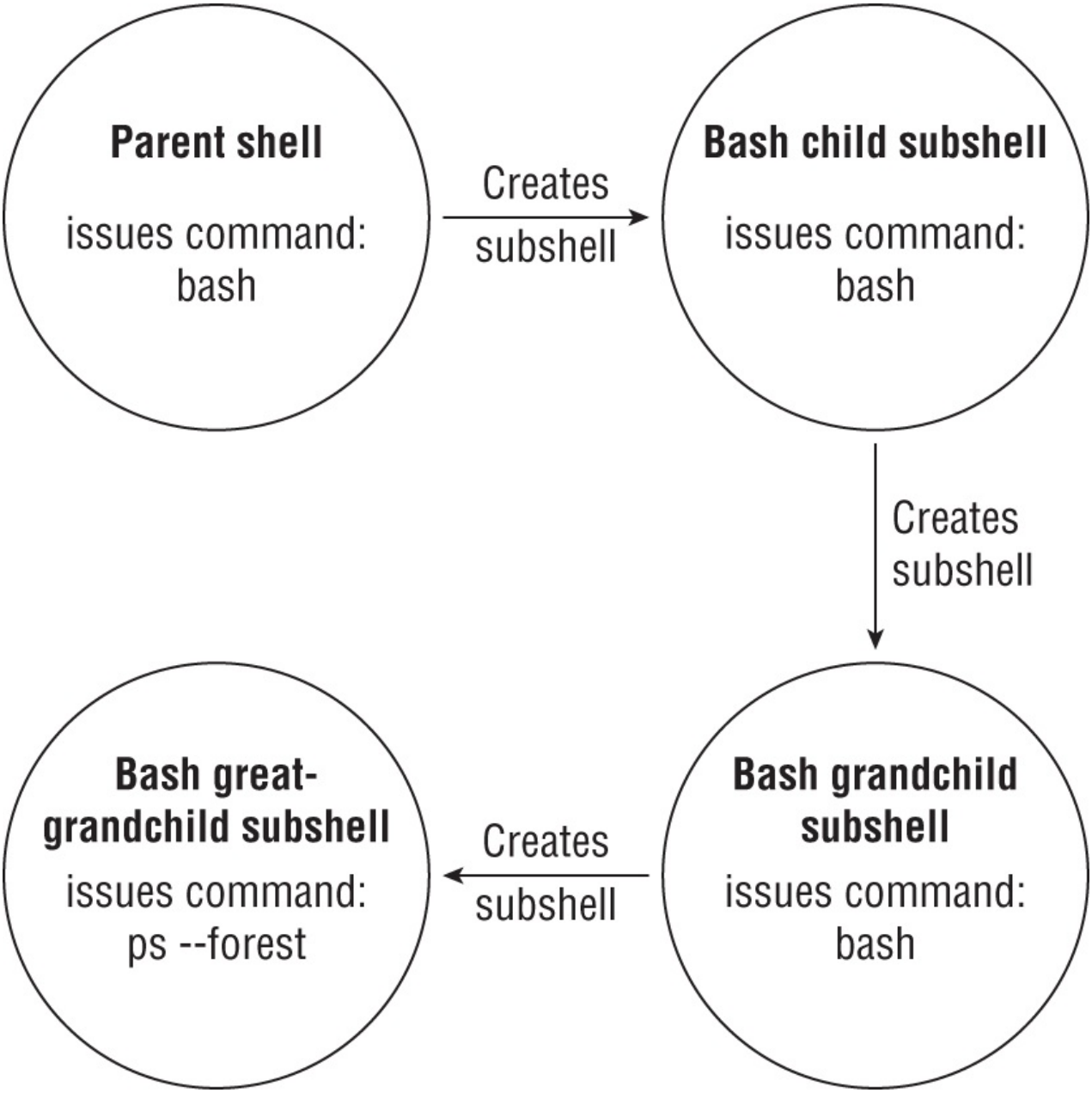

FIGURE 5-2 Subshell nesting

TABLE 5-1 The `bash` Command-Line Options

Option	Description
`-c` *`string`*	Reads commands from *`string`* and processes them
`-i`	Starts an interactive shell, allowing input from the user
`-l`	Acts as if invoked as a login shell
`-r`	Starts a restricted shell, limiting the user to the default directory
`-s`	Reads commands from the standard input

You can find more help on the `bash` command and even more command-line parameters by typing **`man bash`** . The `bash --help` command provides additional assistance as well.

TIP

If you'd like to see the version of the Bash shell, just type bash --version at the command line. This won't create a subshell, but instead displays the current version of your system's GNU Bash shell program.

You can gracefully exit out of each subshell by entering the **exit** command:

```
$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
christi+  7650  7649  0 16:01 pts/0    00:00:00 -bash
christi+  7687  7650  0 16:02 pts/0    00:00:00 bash
christi+  7709  7687  0 16:02 pts/0    00:00:00 bash
christi+  7731  7709  0 16:02 pts/0    00:00:00 bash
christi+  8080  7731  0 16:35 pts/0    00:00:00 ps -f
$
$ exit
exit
$
$ ps --forest
  PID TTY          TIME CMD
 7650 pts/0    00:00:00 bash
 7687 pts/0    00:00:00  \_ bash
 7709 pts/0    00:00:00      \_ bash
 8081 pts/0    00:00:00          \_ ps
$
$ exit
exit
$ exit
exit
$
$ ps --forest
  PID TTY          TIME CMD
 7650 pts/0    00:00:00 bash
 8082 pts/0    00:00:00  \_ ps
$
```

Not only does the `exit` command allow you to leave child subshells, but you can log out of your current virtual console terminal or terminal emulation software as well. Just type **exit** in the parent shell, and you gracefully exit the CLI.

Also, a subshell is sometimes created when you run a shell script. You learn more about that topic in Chapter 11. Next we'll cover how a subshell is spawned using a process list.

Looking at process lists

On a single line, you can designate a list of commands to be run one after another. This is done by entering a command list using a semicolon (`;`) between the commands:

```
$ pwd ; ls test* ; cd /etc ; pwd ; cd ; pwd ; ls my*
/home/christine
test_file  test_one  test_two
/etc
/home/christine
my_file  my_scrapt  my_script  my_scrypt
$
```

In the preceding example, the commands all executed one after another with no problems. However, while using commands in this manner is called a list, it is not a *process list*. For a command list to be considered a process list, the commands must be encased in parentheses:

```
$ (pwd ; ls test* ; cd /etc ; pwd ; cd ; pwd ; ls my*)
/home/christine
test_file  test_one  test_two
/etc
/home/christine
my_file  my_scrapt  my_script  my_scrypt
$
```

Though the parentheses addition may not appear to be a big difference, they do cause a very different effect. Adding parentheses and turning the command list into a process list created a subshell to execute the commands.

> **NOTE**
>
> **A process list is a command grouping type. Another command grouping type puts the commands between curly brackets and ends the command list with a semicolon (;). The syntax is as follows: `{ command; }`. Using curly brackets for command grouping does not create a subshell as a process list does.**

To indicate if a subshell was spawned, a command using an environment variable is needed here. (Environment variables are covered in detail in Chapter 6). The command needed is `echo $BASH_SUBSHELL`. If it returns 0, then there is no subshell. If it returns 1 or more, a subshell was created.

First, the example using just a command list is executed with the `echo $BASH_SUBSHELL` tacked onto the end:

```
$ pwd ; ls test* ; cd /etc ; pwd ; cd ; pwd ; ls my* ; echo $BASH_SUBSHELL
/home/christine
test_file  test_one  test_two
/etc
/home/christine
my_file  my_scrapt  my_script  my_scrypt
0
$
```

At the very end of the commands' output, you can see the number zero (`0`) is displayed. This indicates a subshell was not created to execute these commands.

The results are different using a process list. The list is executed with `echo $BASH_SUBSHELL` tacked onto the end:

```
$ (pwd ; ls test* ; cd /etc ; pwd ; cd ; pwd ; ls my* ; echo $BASH_SUBSHELL)
/home/christine
test_file  test_one  test_two
/etc
/home/christine
my_file  my_scrapt  my_script  my_scrypt
1
$
```

In this case, the number one (`1`) displayed at the output's end. This indicates a subshell was indeed created and used for executing these commands.

Thus, a process list is a command grouping enclosed with parentheses, which creates a subshell to execute the command(s). You can even create a grandchild subshell by embedding parentheses within a process list:

```
$ (pwd ; echo $BASH_SUBSHELL)
/home/christine
1
$ (pwd ; (echo $BASH_SUBSHELL))
/home/christine
2
$
```

Notice in the first process list, the number one (`1`) is displayed, indicating a child subshell as you would expect. However, in the example's second process list, additional parentheses were included around the `echo $BASH_SUBSHELL` command. These additional parentheses caused a grandchild subshell to be created for the command's execution. Thus, a number two (`2`) was displayed, indicating a subshell within a subshell.

Subshells are often used for multiprocessing in shell scripts. However, entering a subshell is an expensive method. (In this situation, *expensive* means that more resources, such as memory and processing power, are consumed.) It can also significantly slow down completion of the task. Subshell issues exist also for an interactive CLI shell session, which is not truly multiprocessing, because the terminal gets tied up with the subshell's I/O.

Creatively using subshells

At the interactive shell CLI, you have more productive ways to use subshells. Process lists, co-processes, and pipes (covered in Chapter 11) all use subshells. Each can be used effectively within the interactive shell.

One productive subshell method in the interactive shell uses background mode. Before we discuss how to use background mode and subshells together, you need to understand background mode.

Investigating background mode

Running a command in background mode allows the command to be processed and frees up your CLI for other use. A classic command to demonstrate background mode is the `sleep` command.

The `sleep` command accepts as a parameter the number of seconds you want the process to wait (sleep). This command is often used to introduce pauses in shell scripts. The command `sleep 10` causes the session to pause for 10 seconds and then return a shell CLI prompt:

```
$ sleep 10
$
```

To put a command into background mode, the `&` character is tacked onto its end. Putting the `sleep` command into background mode allows a little investigation with the `ps` command:

```
$ sleep 3000&
[1] 2542
$
$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
christi+  2356  2352  0 13:27 pts/0    00:00:00 -bash
christi+  2542  2356  0 13:44 pts/0    00:00:00 sleep 3000
christi+  2543  2356  0 13:44 pts/0    00:00:00 ps -f
$
```

The `sleep` command was told to sleep for `3000` seconds (50 minutes) in the background (`&`). When it was put into the background, two informational items were displayed before the shell CLI prompt was returned. The first informational item is the background job's number (`1`) displayed in brackets. The second item is the background job's process ID (`2542`).

The `ps` command was used to display the various processes. Notice that the `sleep 3000` command is listed. Also note that its PID in the second column is the same PID displayed when the command went into the background, `2542`.

In addition to the `ps` command, you can use the `jobs` command to display background job information. The `jobs` command displays your processes (jobs) currently running in background mode:

```
$ jobs
[1]+  Running                 sleep 3000 &
$
```

The `jobs` command shows the job number (`1`) in brackets. It also displays the job's current status (`Running`) as well as the command itself (`sleep 3000 &`).

You can see even more information by using the `-l` (lowercase L) parameter on the `jobs` command. The `-l` parameter displays the command's PID in addition to the other information:

```
$ jobs -l
[1]+  2542 Running                 sleep 3000 &
$
```

TIP

When you have more than one background process running, there is some additional helpful information to show which background job was started last. The most recently started job has a plus sign (+) next to its job number in the `jobs` command's display. And the second newest process will have a minus sign (-) to provide you with additional information.

When the background job is finished, its completion status is displayed the next time you press the Enter key at the command line:

```
$
[1]+  Done                    sleep 3000
$
```

Background mode is very handy. And it provides a method for creating useful subshells at the CLI.

Putting process lists into the background

By placing process lists into the background, you can do large amounts of multiprocessing within a subshell. A side benefit is that your terminal is not tied up with the subshell's I/O.

As stated earlier, a process list is a command or series of commands executed within a subshell. Using a process list including `sleep` commands and displaying the `BASH_SUBSHELL` variable operates as you would expect:

```
$ (sleep 2 ; echo $BASH_SUBSHELL ; sleep 2)
1
$
```

In the preceding example, a two-second pause occurs, the number one (`1`) is displayed indicating a single subshell level (child shell), and then another two-second pause occurs before the prompt returns. Nothing too dramatic here.

Putting the same process list into background mode can cause a slightly different effect with command output:

```
$ (sleep 2 ; echo $BASH_SUBSHELL ; sleep 2)&
[1] 2553
$ 1

[1]+  Done                    ( sleep 2; echo $BASH_SUBSHELL; sleep 2 )
$
```

Putting the process list into the background causes a job number and process ID to appear, and the prompt returns. However, the odd event is that the displayed number one (`1`), indicating a single-level subshell, is displayed next to the prompt! Don't let this confuse you. Simply press the Enter key, and you get another prompt back.

Using a process list in background mode is one creative method for using subshells at the CLI. This allows you to be more productive with fewer keystrokes.

Of course, the process list of `sleep` and `echo` commands are just for example purposes. Creating backup files with `tar` (see Chapter 4) is a more practical example of using background process lists effectively:

```
$ (tar -cf Doc.tar Documents ; tar -cf Music.tar Music)&
[1] 2567
$
$ ls *.tar
Doc.tar  Music.tar
[1]+  Done                    ( tar -cf Doc.tar Documents;
 tar -cf Music.tar Music )
$
```

Putting a process list in background mode is not the only way to use subshells creatively at the CLI. Co-processing is another method.

Looking at co-processing

Co-processing does two things at the same time. It spawns a subshell in background mode and it executes a command within that subshell.

To perform co-processing, the `coproc` command is used along with the command to be executed in the subshell:

```
$ coproc sleep 10
[1] 2689
$
```

Co-processing performs almost identically to putting a command in background mode, except for the fact that it creates a subshell. You'll notice that when the `coproc` command and its parameters were entered, a background job was started. The background job number (`1`) and process ID (`2689`) were displayed on the screen.

The `jobs` command allows you to display the co-processing status:

```
$ jobs
[1]+  Running                 coproc COPROC sleep 10 &
$
```

From the preceding example, you can see that the background command executing in the subshell is `coproc COPROC sleep 10` . `COPROC` is a name given to the process by the `coproc` command. You can set the name yourself by using extended syntax for the command:

```
$ coproc My_Job { sleep 10; }
[1] 2706
$
$ jobs
[1]+  Running                 coproc My_Job { sleep 10; } &
$
```

By using the extended syntax, the co-processing name was set to `My_Job` . Be careful here, because the extended syntax is a little tricky. Make sure you place a space after the first curly bracket (`{`) and before the start of your command. Also, the command must end with a semicolon (`;`). And you must put a space after the semicolon and before the closing curly bracket (`}`).

> **NOTE**
>
> **Co-processing allows you to get very fancy and send/receive information to the process running in the subshell. The only time you need to name a co-process is when you have multiple co-processes running and you need to communicate with them all. Otherwise, just let the `coproc` command set the name to the default, `COPROC`.**

You can be really clever and combine co-processing with process lists, creating nested subshells. Just type your process list and put the command `coproc` in front of it:

```
$ coproc ( sleep 10; sleep 2 )
[1] 2750
$
$ jobs
[1]+  Running                 coproc COPROC ( sleep 10; sleep 2 ) &
$
$ ps --forest
  PID TTY          TIME CMD
 2367 pts/0    00:00:00 bash
 2750 pts/0    00:00:00  \_ bash
 2751 pts/0    00:00:00  |   \_ sleep
 2752 pts/0    00:00:00  \_ ps
$
```

Just remember that spawning a subshell can be expensive and slow. Creating nested subshells is even more so!

Using subshells can provide flexibility as well as convenience. Understanding their behavior is important in order to

reach these goals. Command actions are also important to understand. In the next section, the behavior differences between built-in and external commands are explored.

Understanding External and Built-In Commands

While learning about the GNU Bash shell, you likely have heard the term *built-in command*. It is important to understand both shell built-in and non–built-in (external) commands. Built-in commands and non–built-in commands operate very differently.

Looking at external commands

An *external command*, sometimes called a filesystem command, is a program that exists outside of the Bash shell. In other words, it is not built into the shell program. An external command program is typically located in `/bin` , `/usr/bin` , `/sbin` , or `/usr/sbin` directories.

The `ps` command is an external command. You can find its filename by using both the `which` and the `type` commands:

```
$ which ps
/usr/bin/ps
$
$ type ps
ps is /usr/bin/ps
$
$ ls -l /usr/bin/ps
-rwxr-xr-x. 1 root root 142216 May 11  2019 /usr/bin/ps
$
```

Whenever an external command is executed, a child process is created. This action is termed *forking*. Conveniently, the external command `ps` displays its current parent as well as its own forked child processes:

```
$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
christi+  2367  2363  0 10:47 pts/0    00:00:00 -bash
christi+  4242  2367  0 13:48 pts/0    00:00:00 ps -f
$
```

Because it is an external command, when the `ps` command executes, a child process is created. In this case, the `ps` command's PID is `4242` and the parent PID is `2367` . The Bash shell process, which in this case is the parent process, has a PID of `2367` . Figure 5-3 illustrates the forking that occurs when an external command is executed.

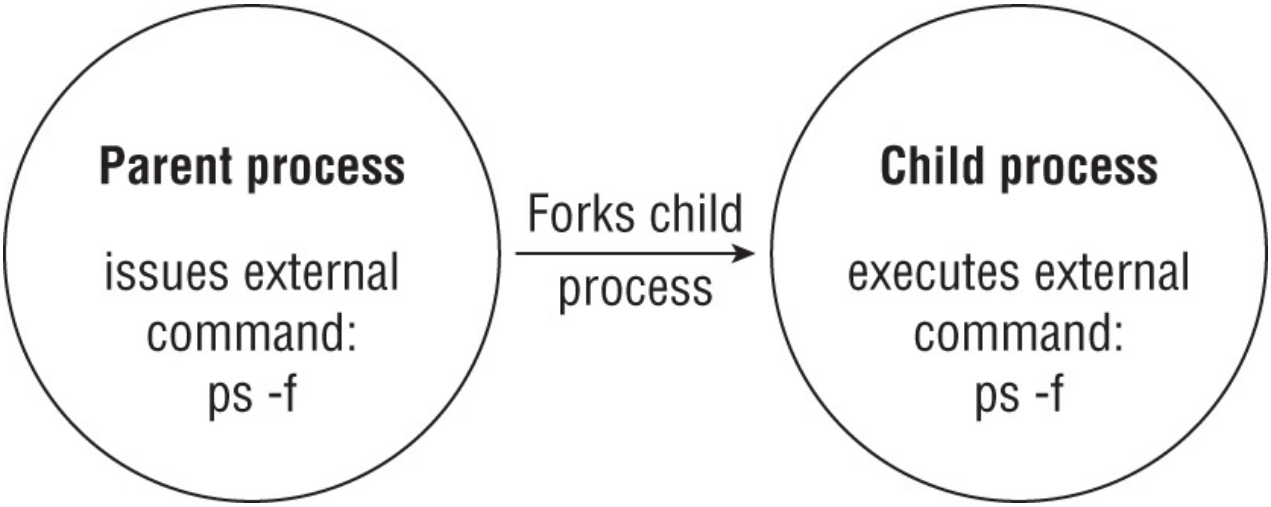

FIGURE 5-3 External command forking

Whenever a process must fork, it takes time and effort to set up the new child process's environment. Thus, external commands can be a little expensive.

> **NOTE**
>
> **If you fork a child process or create a subshell, you can still communicate with it via signaling, which is extremely helpful in both the command line and in writing shell scripts. Signaling allows process communication via signals. Signals and signaling are covered in Chapter 16, "Script Control."**

When using a built-in command, no forking is required. Therefore, built-in commands are less expensive.

Looking at built-in commands

Built-in commands are different in that they do not need a child process to execute. They were compiled into the shell, and thus are part of the shell's toolkit. No external program file exists to run them.

Both the `cd` and `exit` commands are built into the Bash shell. You can tell a command is built-in by using the `type` command:

```
$ type cd
cd is a shell builtin
$
$ type exit
exit is a shell builtin
$
```

Because they do not need to fork a child process to execute or open a program file, built-in commands are faster and more efficient. A list of GNU Bash shell built-in commands is provided in Appendix A.

Be aware that some commands have multiple flavors. For example, both `echo` and `pwd` have a built-in command flavor as well as an external command flavor. These flavors are slightly different. To see multiple flavors for commands, use the `-a` option on the `type` command:

```
$ type -a echo
echo is a shell builtin
echo is /usr/bin/echo
$
$ which echo
/usr/bin/echo
$
$ type -a pwd
pwd is a shell builtin
pwd is /usr/bin/pwd
$
$ which pwd
/usr/bin/pwd
$
```

Using the `type -a` command shows both types (built-in and external) for each of the two commands. Note that the `which` command shows only the external command file.

TIP

To use the external command for a command that has multiple flavors, directly reference the file. For example, to use the `pwd` external command, type /usr/bin/pwd.

Using the *history* command

The Bash shell keeps track of the most recent commands you have used. You can recall these commands and even reuse them. A helpful built-in command that lets you explore and manage these previously issued commands is the `history` command.

To see a list of recently used commands, type the `history` command with no options:

```
$ history
    1  ps -f
    2  pwd
    3  ls
    4  coproc ( sleep 10; sleep 2 )
    5  jobs
    6  ps --forest
    7  ls
    8  ps -f
    9  pwd
   10  ls -l /usr/bin/ps
   11  history
   12  cd /etc
   13  pwd
   14  ls
   15  cd
   16  type -a pwd
   17  which pwd
   18  type -a echo
   19  which echo
   20  ls
[...]
$
```

In this example, the listing is snipped and only the first 20 commands are shown. Typically, the last 1,000 commands are kept in history. That's a lot of commands!

> **TIP**
>
> **You can set the number of commands to keep in the Bash history. To do so, you need to modify an environment variable called `HISTSIZE` (see Chapter 6).**

You can recall and reuse the last command in your history list. This can save time and typing. To recall and reuse your last command, type `!!` and press the Enter key:

```
$ ps --forest
  PID TTY          TIME CMD
 2367 pts/0    00:00:00 bash
 5240 pts/0    00:00:00  \_ ps
$
$ !!
ps --forest
  PID TTY          TIME CMD
 2367 pts/0    00:00:00 bash
 5241 pts/0    00:00:00  \_ ps
$
```

When `!!` was entered, the Bash shell first displayed the command it was recalling from the shell's history. And after the command was displayed, it was executed.

Command history is kept in the hidden `.bash_history` file, which is located in each user's home directory:

```
$ pwd
/home/christine
$
$ ls .bash_history
.bash_history
$
```

Be aware that during your CLI session, the `bash` command history is stored in memory. It is only written out into the history file when the shell is exited:

```
$ history
    1  ps -f
    2  pwd
[...]
   38  exit
   39  history
   40  ps --forest
   41  ps --forest
   42  pwd
   43  ls .bash_history
   44  history
$
$ cat .bash_history
ps -f
pwd
[...]
ls
history
exit
$
```

Notice that when the `history` command is run, the last commands displayed do not match final commands in the `.bash_history` file. There were six additional commands issued, which are not recorded in the history file.

You can force the command history to be written to the `.bash_history` file without leaving a shell session. In order to impose this write, use the `-a` option on the `history` command:

```
$ history -a
$
$ history
    1  ps -f
    2  pwd
[...]
   38  exit
   39  history
   40  ps --forest
   41  ps --forest
   42  pwd
   43  ls .bash_history
   44  history
   45  cat .bash_history
   46  history -a
   47  history
$
$ cat .bash_history
ps -f
pwd
[...]
```

```
exit
history
ps --forest
ps --forest
pwd
ls .bash_history
history
cat .bash_history
history -a
$
```

Notice that contents from both the `history` command and the `.bash_history` file match, except for the very last command listed (the `history` command), because it came after the `history -a` command was issued.

> **NOTE**
>
> **If you have multiple terminal sessions open, you can still append `.bash_history` in each open session using the `history -a` command. However, the histories are not automatically updated for your other open terminal sessions. This is because the `.bash_history` file is read-only when a terminal session is first started. To force the `.bash_history` file to be reread and a terminal session's history in memory to be updated, use the `history -n` command.**

You can recall any command from the history list. Just enter an exclamation point and the command's number from the history list:

```
$ history
    1  ps -f
    2  pwd
[...]
   39  history
   40  cat .bash_history
   41  ps --forest
   42  pwd
   43  ps -f
   44  history
   45  cat .bash_history
   46  history -a
   47  history
   48  cat .bash_history
   49  history
$
$ !42
pwd
/home/christine
$
```

Command number 42 was pulled from the history list. Notice that similar to executing the last command in history, the Bash shell first displays the command it is recalling from the shell's history. After the command is displayed, it is executed.

> **TIP**
>
> **If for some reason you need to clear out command history, it's simple to do. Just type history -c and the contents of the current history are wiped. Next enter history -a to also clear out the `.bash_history` file.**

Using Bash shell command history can be a great timesaver. You can do even more with the built-in `history` command. Be sure to view the Bash manual pages for `history` by typing **`man history`** .

Using command aliases

The `alias` command is another useful built-in shell command. A *command alias* allows you to create an alias name for common commands (along with their parameters) to help keep your typing to a minimum.

Most likely, your Linux distribution has already set some common command aliases for you. To see a list of the active aliases, use the `alias` command along with the `-p` parameter:

```
$ alias -p
[...]
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
$
```

Notice that, on this Ubuntu Linux distribution, an alias is used to override the standard `ls` command. It

automatically provides the `--color=auto` parameter, which will cause the `ls` command to use color coding (for example, directories may be shown in blue), if the terminal supports colorization. The `LS_COLORS` environment variable controls the color codes used (environment variables are covered in Chapter 6).

> **TIP**
>
> **When jumping between various distributions, be cautious using color coding to help you determine which listed name is a directory and which is a file. Because color codes are not standardized, it's best to use the `ls -F` command to see the file's type code instead.**

You can create your own aliases using the `alias` command:

```
$ alias li='ls -i'
$
$ li
34665652 Desktop          1415018 NetworkManager.conf
 1414976 Doc.tar         50350618 OldDocuments
34665653 Documents        1414981 Pictures
51693739 Downloads       16789591 Public
 1415016 hlink_test_one   1415019 really_ridiculously_long_file_name
 1415021 log_file         1415020 slink_test_file
51693757 Music            1415551 Templates
 1414978 Music.tar        1415523 test_file
 1415525 my_file          1415016 test_one
 1415524 my_scrapt        1415017 test_two
 1415519 my_script       16789592 Videos
 1415015 my_scrypt
$
```

After you define an alias value, you can use it at any time in your shell, including in shell scripts. Be aware that because command aliases are built-in commands, an alias is valid only for the shell process in which it is defined:

```
$ alias li='ls -i'
$
$ bash
$ li
bash: li: command not found...
$
$ exit
exit
$
$ li
34665652 Desktop          1415018 NetworkManager.conf
 1414976 Doc.tar         50350618 OldDocuments
[...]
1415524 my_scrapt        1415017 test_two
 1415519 my_script       16789592 Videos
 1415015 my_scrypt
$
```

> **TIP**
>
> **If needed, you can turn off an alias by typing unalias alias-name at the command line. Keep in mind, if the alias wasn't set by you, it will be turned back on the next time you log into the system. You can modify your environment files to permanently disable an alias. Environment files are covered in Chapter 6.**

Fortunately, you can make an alias value permanent across subshells. The next chapter covers how to do that, along with environment variables.

Summary

This chapter discussed the complicated interactive program, the GNU Bash shell. We covered understanding the shell process and its relationships, including how subshells are spawned and their relationship to the parent shell. We also explored commands that create child processes and commands that don't.

The default interactive shell is normally started whenever a user logs into a terminal. The shell that the system starts depends on a user ID configuration. Typically, it is `/usr/bin/bash` . The default system shell, `/usr/bin/sh` , is used for system shell scripts, such as those needed at startup.

A subshell or child shell can be spawned using the `bash` command. They are also created when a process list or the `coproc` command is used. Using subshells at the command line can allow for creative and productive use of the CLI. Subshells can be nested, spawning grandchild shells and great-grandchild shells. Creating a subshell is an expensive process because a new environment for the shell must be created as well.

Finally, we looked at two different types of shell commands: built-in and external commands. External commands

create a child process with a new environment, but a built-in command does not. This causes external commands to be more expensive to use. Because a new environment is not needed, built-in commands are more efficient and not affected by any environment changes.

Shells, subshells, processes, and forked processes are all affected by environment variables. How the variables affect and can be used within these different contexts is explored in the next chapter.